

UCS503 - Software Engineering

Willow

**An Image-Based Harmful-Ingredient Detection System
for the Indian Market**

Prepared by:

[1024160001] Aditya Gupta
[1024160019] Akanskha Thakur

Group: [3p11] | BE Third Year

Submitted to: Jalani Asif

Mid-Semester Evaluation

Department of Computer Science and Engineering
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	4
2.2	Implementation Details	4
2.2.1	Module 1: Ingredient Extraction Pipeline	4
2.2.2	Module 2: Flutter Frontend Integration	5
3	Results and Evaluation	6
3.1	Testing Methodology	6
3.2	Performance Metrics	6
3.3	Analysis	7
4	Conclusion	8
4.1	Summary of Work	8
4.2	Key Findings	8
4.3	Future Work and Recommendations	8
4.4	Final Remarks	9
	Bibliography	10

Chapter 1

Introduction

1.1 Background and Context

Packaged and locally sold food products in the Indian market frequently carry ingredient information that is incomplete, inconsistent, or entirely absent from a consumer’s point of view. Rising health consciousness among Indian consumers has increased demand for tools that explain what a product actually contains, yet the tools available today are not built for this market. Global ingredient-scanning applications such as Yuka [1] rely on barcode-to-database lookups and score ingredients against EU/US additive standards, leaving unbranded, regional, and small-vendor Indian products – a large share of the market – effectively unscanned.

This gap is not hypothetical. In April 2024, Gujarat’s Food and Drug Control Administration confiscated over 60,000 kg of adulterated spices, including chilli powder, turmeric, and coriander powder, in a single enforcement drive [4]. That same month, Hong Kong and Singapore both suspended sales of spice-mix products from two major Indian brands, MDH and Everest, after detecting the carcinogenic pesticide ethylene oxide beyond permissible limits [5]. Food safety and ingredient transparency are therefore live, documented concerns in the Indian market, not a theoretical justification for this project.

1.1.1 Problem Statement

Existing ingredient-scanning solutions available to Indian consumers are inadequate for three concrete reasons:

1. **Barcode dependency excludes local products.** Applications that rely on barcode-to-database lookups fail systematically for unbranded and regional Indian products, which are rarely barcode-indexed in the first place.
2. **Many local products carry no printed ingredient list at all.** Local sweets, bakery items, and loose-packaged snacks are common in the Indian market and frequently have no ingredient panel to read.
3. **Scoring standards are not localized.** Existing tools apply EU/US additive standards rather than FSSAI guidelines [2], and do not account for allergens or ingredients specific to Indian diets.

1.2 Project Objectives

1. **Objective 1:** Extract a structured ingredient list from a photograph of a packaged food product, without depending on a barcode lookup, measurably tested against a set of real product photographs.
2. **Objective 2:** Flag ingredients known to be harmful and present their associated health effects in plain, non-technical language, sourced from a curated ingredient database rather than generated live.

3. **Objective 3:** Handle the common case of unbranded or local products with no printed ingredient list by classifying the product into a known category and presenting a clearly labelled, generic risk estimate.
4. **Objective 4:** Suggest healthier alternative products within the same category and a comparable price range.
5. **Objective 5:** Build the system so that new product categories and data sources can be added through configuration rather than a redesign of the extraction or classification pipeline.

These objectives were scoped deliberately to be measurable within a one-semester, two-to-three person team timeline, rather than attempting the full long-term vision (additional categories, crowdsourced data, regional-language support) within the MVP itself.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

The system follows a modular pipeline: *Photo Upload* → *Extraction Service* → *Classification Engine* → *Alternatives Engine* → *Results*. A separate Category Configuration store (categories, ingredients, aliases, harm rules, and alternatives data) feeds the Classification and Alternatives modules, so that category-specific behaviour is data-driven rather than hard-coded into the pipeline logic. This was a deliberate design decision: the Minimum Viable Product (MVP) is scoped to a single category (food), but the architecture is built so that additional categories can be added later through configuration and data alone.

For the current phase, the backend is implemented as a modular monolith – clearly separated internal modules for extraction, classification, and alternatives – rather than deployed microservices, to avoid infrastructure overhead that a one-semester student team cannot absorb, while preserving a clean path to splitting into independent services later if required.

2.1.2 Technical Stack

- **Framework/Language:** Python (Flask, for the backend API layer) and Flutter/Dart (for the mobile frontend).
- **Database:** PostgreSQL is planned for the category configuration store (categories, ingredients, aliases, alternatives); the current extraction-only prototype does not yet require a persistent database.
- **Tools:** Git for version control, VS Code for development, Overleaf/L^AT_EX for documentation, and draw.io for architecture, use-case, ER, and activity diagrams.
- **Libraries:** `google-genai` and `anthropic` (vision-capable extraction APIs), `pytesseract` and `Pillow` (local OCR fallback), `flask-cors` (API layer), and `http/image_picker` (Flutter HTTP client and camera capture).

2.2 Implementation Details

2.2.1 Module 1: Ingredient Extraction Pipeline

The extraction layer is built behind a single abstract interface (`IngredientExtractor`), so that the underlying vision provider can be swapped without any change to downstream code. Three concrete implementations exist at this stage: a local, free Tesseract-based extractor (used for offline development and testing without API cost), a Gemini vision-based extractor, and a Claude vision-based extractor, allowing direct accuracy

and latency comparison between providers on the same input.

A deliberate architectural boundary was drawn early in this module: the vision API's only responsibility is reading text off the image. It does not judge, classify, or explain any ingredient. This decision was made because large language models are prone to subtle inaccuracies on exactly the kind of specific, falsifiable health claims this application makes, and an incorrect claim shown to a real user is a liability concern, not merely a UX one. Classification against a curated, sourced database is therefore treated as a separate module (planned for the next phase), so that every harmful-ingredient flag shown to the user can be traced back to a specific, checkable source rather than to a live model inference.

A parsing layer sits between raw model output and the rest of the system: it splits comma/semicolon-separated ingredient text while correctly preserving nested parenthetical groups (for example, "**Raising Agents** (INS 500(ii))" is treated as a single ingredient, not split at the inner parenthesis), strips common label prefixes such as "INGREDIENTS:", and normalises whitespace. This parser is covered by a unit test suite (Section 3.1) and was verified against a realistic synthetic Indian product label containing INS-coded additives before being used on real photographs.

2.2.2 Module 2: Flutter Frontend Integration

To connect the existing Flutter frontend to the extraction pipeline for the current evaluation milestone, a minimal REST API was built around the extraction module using Flask. A single endpoint, `POST /extract`, accepts a multipart image upload and an optional provider selector, and returns a JSON response containing the parsed ingredient list, the raw extracted text, a confidence score, and the provider used. Error conditions (unsupported file type, missing image field, or a provider-level extraction failure) return a 400 response with a specific, human-readable error message rather than a generic failure.

On the Flutter side, the `image_picker` package is used to capture a photograph directly from the device camera, and the `http` package's `MultipartRequest` is used to upload it to the backend. A practical integration detail worth documenting: the correct backend base URL differs by run target (10.0.2.2 for the Android emulator, 127.0.0.1 for the iOS simulator, and the development machine's LAN IP address for a physical device on the same network) – a common point of confusion when connecting a mobile client to a locally running backend during development.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** The ingredient-list parser is covered by a dedicated unit test suite exercising comma/semicolon splitting, nested parenthetical groups, label-prefix stripping, whitespace normalisation, and empty-input handling.
- **Integration Testing:** The full extraction pipeline (photo $\rightarrow$ extraction $\rightarrow$ parsing) was verified end-to-end against a synthetic product label, and the Flask API layer was verified against the same pipeline using real HTTP requests, including both successful extractions and the two documented error paths.
- **Performance Testing:** A comparison harness was built to run every configured extraction provider (Tesseract, Gemini, Claude) against a shared set of product photographs and report per-provider latency, and, where a manually transcribed ground truth is supplied, precision and recall of the extracted ingredient list.
- **User Testing:** Not yet conducted. A pilot with 10-15 testers, comparing system output against manual expert review of the same product photographs, is planned once the classification engine (Chapter 4) is complete.

3.2 Performance Metrics

The results below are preliminary smoke-test results from the current extraction-only milestone, not the full pilot evaluation, which depends on the classification engine described in Section 4.3.

Metric	Result
Parser unit tests passing	10/10
Ingredients correctly parsed (synthetic label test)	8/8
Precision (synthetic label, deliberate-mismatch check)	1.00
Recall (synthetic label, deliberate-mismatch check)	0.89
API error-handling paths verified	2/2
Real-world accuracy (10-20 photo test set, mixed conditions)	Pending

Table 3.1: Preliminary Extraction Pipeline Results

3.3 Analysis

The objectives met so far are Objective 1 (extraction pipeline, built and unit-tested) and the frontend integration required to demonstrate it end-to-end from the Flutter application. Objectives 2 through 4 (harmful-ingredient flagging, the unbranded-product fallback, and alternative suggestions) depend on the classification engine and category-configuration database, which are the next planned milestone and have not yet been implemented.

Two challenges were encountered and resolved during this phase. First, an early design question was whether the vision API used for extraction should also judge which ingredients are harmful. This was rejected in favour of a strict separation (Section 2.2.1), since a live model judgment cannot be reliably traced to a specific, checkable source for a health-related claim. Second, connecting the Flutter frontend to a locally running backend surfaced the common but under-documented issue of platform-specific base URLs; this was resolved by explicitly documenting the correct host address for each run target rather than assuming a single URL would work across emulator, simulator, and physical-device testing.

The remaining, not-yet-resolved challenge is real-world extraction accuracy: the pipeline has only been verified against a single synthetic label so far. Testing against a genuinely varied set of real product photographs (clear, blurry, curved packaging, and low light) is the immediate next step and is required before any accuracy claim can be made with confidence.

Chapter 4

Conclusion

4.1 Summary of Work

At this mid-semester stage, the project has completed requirements scoping, system architecture design, and a working, tested ingredient extraction pipeline with two interchangeable vision-API providers and a local free-tier fallback, connected end-to-end to the existing Flutter frontend through a minimal REST API. Objective 1 is functionally complete pending real-world accuracy testing; Objectives 2 through 5 are designed but not yet implemented.

4.2 Key Findings

- **Finding 1:** An image-first extraction approach, rather than a barcode-lookup approach, is architecturally necessary to cover unbranded and local Indian products, which make up a significant share of the market existing tools do not reach.
- **Finding 2:** Separating ingredient extraction (a reading task, delegated to a vision API) from harmful-ingredient classification (a judgment task, delegated to a curated, sourced database) removes a specific and serious hallucination risk that a single-model approach would carry on health-related claims.
- **Finding 3:** Practical mobile-to-backend integration details, such as platform-specific base URLs, are a recurring but often undocumented source of friction when connecting a Flutter client to a locally hosted development server, and are worth documenting explicitly for the rest of the team.

4.3 Future Work and Recommendations

1. Build the classification engine and the category-configuration database (categories, curated ingredients, aliases, and harm rules), seeded from Open Food Facts [3] and cross-referenced against the FSSAI Compendium on Food Safety and Standards (Food Additives) Regulation [2].
2. Implement the category-based fallback for unbranded products with no printed ingredient list, including the generic risk-profile disclaimer described in the project’s requirements documentation.
3. Build the healthier-alternatives engine and the full results screen in the Flutter application.
4. Conduct the planned pilot test (10-15 users) and collect the evaluation metrics defined for this project: extraction accuracy, flagging precision, and time-to-result.
5. Stretch goals, to be attempted only if the core scope above is complete with time remaining: a second product category (cosmetics and personal care), regional-language OCR support, and integration with the FSSAI FoSCoS license lookup as an additional trust signal for unbranded products.

4.4 Final Remarks

The mid-semester phase of this project has validated the core technical approach – image-based extraction with a provider-agnostic interface – against real code and real (if preliminary) test results, rather than design alone. The most significant remaining risk is not the extraction pipeline itself, which is functioning, but the accuracy of extraction across the genuinely messy conditions of real Indian product packaging, which has not yet been measured. Closing that gap, and building the classification engine that turns an extracted ingredient list into an actionable, trustworthy health signal for the user, are the primary goals for the remainder of the semester.

Bibliography

- [1] Yuka. *Yuka*. <https://yuka.io>. Accessed 16 September 2026.
- [2] Food Safety and Standards Authority of India. *Compendium on Food Safety and Standards (Food Products Standards and Food Additives) Regulation*. 2011. <https://fssai.gov.in/cms/Compendium-FSS-FPS-FA.php>. Accessed 16 September 2026.
- [3] Open Food Facts. *Open Food Facts — Data, API and SDKs*. <https://world.openfoodfacts.org/data>. Accessed 16 September 2026.
- [4] Daijiworld. *Gujarat: Food Safety Raid Uncovers 1,500 kg of Adulterated Paneer in Ahmedabad*. 2025. <https://www.daijiworld.com/index.php/news/newsDisplay?newsID=1268855>. Article reports the April 2024 Gujarat FDCA seizure of over 60,000 kg of adulterated spices. Accessed 16 September 2026.
- [5] Deccan Herald. *After Singapore, Hong Kong Bans Everest Fish Curry Masala, MDH Spices*. 2024. <https://www.deccanherald.com/business/companies/after-singapore-hong-kong-bans-everest-fish-curry-masala-mdh-spices-2988741>. Accessed 16 September 2026.